【K1】数值和编码作业

题目1：十进制转二进制

题目描述

将下列十进制数（含小数）转换为二进制数： - 57，128，12.5，7.198，3972，1.35，1000

代码实现

```
def decimal_to_binary(num):
    if num == 0:
        return "0"

    sign = ""
    if num < 0:
        sign = "-"
        num = abs(num)

    integer_part = int(num)
    fractional_part = num - integer_part

    binary_integer = []
    if integer_part > 0:
        temp = integer_part
        print(f"  整数部分转换 ({integer_part} → 二进制):")
        while temp > 0:
            remainder = temp % 2
            binary_integer.append(str(remainder))
            print(f"    {temp} ÷ 2 = {temp // 2} ... 余数 {remainder}")
            temp = temp // 2
        binary_integer.reverse()
    else:
        binary_integer = ["0"]

    binary_fractional = []
    if fractional_part > 0:
        temp = fractional_part
        print(f"  小数部分转换 ({fractional_part:.6f} → 二进制):")
        precision = 20
        count = 0
        while temp > 0 and count < precision:
            original = temp
            temp *= 2
            integer = int(temp)
            binary_fractional.append(str(integer))
            print(f"    {original:.6f} × 2 = {temp:.6f} ... 整数 {integer}")
            temp -= integer
            count += 1

    result = sign + "".join(binary_integer)
    if binary_fractional:
        result += "." + "".join(binary_fractional)

    return result
```

运行结果

```
【十进制】57
  整数部分转换 (57 → 二进制):
    57 ÷ 2 = 28 ... 余数 1
    28 ÷ 2 = 14 ... 余数 0
    14 ÷ 2 = 7 ... 余数 0
    7 ÷ 2 = 3 ... 余数 1
    3 ÷ 2 = 1 ... 余数 1
    1 ÷ 2 = 0 ... 余数 1
【二进制】111001

【十进制】128
  整数部分转换 (128 → 二进制):
    128 ÷ 2 = 64 ... 余数 0
    64 ÷ 2 = 32 ... 余数 0
    32 ÷ 2 = 16 ... 余数 0
    16 ÷ 2 = 8 ... 余数 0
    8 ÷ 2 = 4 ... 余数 0
    4 ÷ 2 = 2 ... 余数 0
    2 ÷ 2 = 1 ... 余数 0
    1 ÷ 2 = 0 ... 余数 1
【二进制】10000000

【十进制】12.5
  整数部分转换 (12 → 二进制):
    12 ÷ 2 = 6 ... 余数 0
    6 ÷ 2 = 3 ... 余数 0
    3 ÷ 2 = 1 ... 余数 1
    1 ÷ 2 = 0 ... 余数 1
  小数部分转换 (0.500000 → 二进制):
    0.500000 × 2 = 1.000000 ... 整数 1
【二进制】1100.1

【十进制】7.198
  整数部分转换 (7 → 二进制):
```

```
    7 ÷ 2 = 3 ... 余数 1
    3 ÷ 2 = 1 ... 余数 1
    1 ÷ 2 = 0 ... 余数 1
  小数部分转换 (0.198000 → 二进制):
    0.198000 × 2 = 0.396000 ... 整数 0
    0.396000 × 2 = 0.792000 ... 整数 0
    0.792000 × 2 = 1.584000 ... 整数 1
    0.584000 × 2 = 1.168000 ... 整数 1
    0.168000 × 2 = 0.336000 ... 整数 0
    0.336000 × 2 = 0.672000 ... 整数 0
    0.672000 × 2 = 1.344000 ... 整数 1
    0.344000 × 2 = 0.688000 ... 整数 0
    0.688000 × 2 = 1.376000 ... 整数 1
    0.376000 × 2 = 0.752000 ... 整数 0
    0.752000 × 2 = 1.504000 ... 整数 1
    0.504000 × 2 = 1.008000 ... 整数 1
    0.008000 × 2 = 0.016000 ... 整数 0
    0.016000 × 2 = 0.032000 ... 整数 0
    0.032000 × 2 = 0.064000 ... 整数 0
    0.064000 × 2 = 0.128000 ... 整数 0
    0.128000 × 2 = 0.256000 ... 整数 0
    0.256000 × 2 = 0.512000 ... 整数 0
    0.512000 × 2 = 1.024000 ... 整数 1
    0.024000 × 2 = 0.048000 ... 整数 0
【二进制】111.00110010101100000010

【十进制】3972
  整数部分转换 (3972 → 二进制):
    3972 ÷ 2 = 1986 ... 余数 0
    1986 ÷ 2 = 993 ... 余数 0
    993 ÷ 2 = 496 ... 余数 1
    496 ÷ 2 = 248 ... 余数 0
    248 ÷ 2 = 124 ... 余数 0
    124 ÷ 2 = 62 ... 余数 0
    62 ÷ 2 = 31 ... 余数 0
    31 ÷ 2 = 15 ... 余数 1
    15 ÷ 2 = 7 ... 余数 1
    7 ÷ 2 = 3 ... 余数 1
    3 ÷ 2 = 1 ... 余数 1
    1 ÷ 2 = 0 ... 余数 1
【二进制】111110000100

【十进制】1.35
  整数部分转换 (1 → 二进制):
    1 ÷ 2 = 0 ... 余数 1
  小数部分转换 (0.350000 → 二进制):
    0.350000 × 2 = 0.700000 ... 整数 0
    0.700000 × 2 = 1.400000 ... 整数 1
    0.400000 × 2 = 0.800000 ... 整数 0
    0.800000 × 2 = 1.600000 ... 整数 1
    0.600000 × 2 = 1.200000 ... 整数 1
    0.200000 × 2 = 0.400000 ... 整数 0
    0.400000 × 2 = 0.800000 ... 整数 0
    0.800000 × 2 = 1.600000 ... 整数 1
    0.600000 × 2 = 1.200000 ... 整数 1
    0.200000 × 2 = 0.400000 ... 整数 0
    0.400000 × 2 = 0.800000 ... 整数 0
    0.800000 × 2 = 1.600000 ... 整数 1
    0.600000 × 2 = 1.200000 ... 整数 1
    0.200000 × 2 = 0.400000 ... 整数 0
    0.400000 × 2 = 0.800000 ... 整数 0
    0.800000 × 2 = 1.600000 ... 整数 1
    0.600000 × 2 = 1.200000 ... 整数 1
    0.200000 × 2 = 0.400000 ... 整数 0
    0.400000 × 2 = 0.800000 ... 整数 0
    0.800000 × 2 = 1.600000 ... 整数 1
【二进制】1.01011001100110011001

【十进制】1000
  整数部分转换 (1000 → 二进制):
    1000 ÷ 2 = 500 ... 余数 0
    500 ÷ 2 = 250 ... 余数 0
    250 ÷ 2 = 125 ... 余数 0
    125 ÷ 2 = 62 ... 余数 1
    62 ÷ 2 = 31 ... 余数 0
```

```
    31 ÷ 2 = 15 ... 余数 1
    15 ÷ 2 = 7 ... 余数 1
    7 ÷ 2 = 3 ... 余数 1
    3 ÷ 2 = 1 ... 余数 1
    1 ÷ 2 = 0 ... 余数 1
【二进制】1111101000
```

题目2：二进制转十进制

题目描述

将下列二进制数（含小数）转换为十进制数： - 11010，110，11.101，0.1011，111.11，111111

代码实现

```python
def binary_to_decimal(binary_str):
    if not binary_str:
        return 0

    sign = 1
    if binary_str.startswith("-"):
        sign = -1
        binary_str = binary_str[1:]

    if "." in binary_str:
        integer_part_str, fractional_part_str = binary_str.split(".", 1)
    else:
        integer_part_str = binary_str
        fractional_part_str = ""

    decimal_integer = 0
    if integer_part_str:
        print(f"  整数部分转换 ({integer_part_str} → 十进制):")
        n = len(integer_part_str)
        for i in range(n):
            digit = int(integer_part_str[i])
            exponent = n - 1 - i
            value = digit * (2 ** exponent)
            decimal_integer += value
            print(f"    {digit} × 2^{exponent} = {value}")
        print(f"    整数部分总和: {decimal_integer}")

    decimal_fractional = 0
    if fractional_part_str:
        print(f"  小数部分转换 ({fractional_part_str} → 十进制):")
        n = len(fractional_part_str)
        for i in range(n):
            digit = int(fractional_part_str[i])
            exponent = -(i + 1)
            value = digit * (2 ** exponent)
            decimal_fractional += value
            print(f"    {digit} × 2^{exponent} = {value}")
        print(f"    小数部分总和: {decimal_fractional:.6f}")

    return sign * (decimal_integer + decimal_fractional)
```

运行结果

```
【二进制】11010
  整数部分转换 (11010 → 十进制):
    1 × 2^4 = 16
    1 × 2^3 = 8
    0 × 2^2 = 0
    1 × 2^1 = 2
    0 × 2^0 = 0
    整数部分总和: 26
【十进制】26

【二进制】110
  整数部分转换 (110 → 十进制):
    1 × 2^2 = 4
    1 × 2^1 = 2
    0 × 2^0 = 0
    整数部分总和: 6
【十进制】6

【二进制】11.101
  整数部分转换 (11 → 十进制):
    1 × 2^1 = 2
    1 × 2^0 = 1
    整数部分总和: 3
  小数部分转换 (101 → 十进制):
    1 × 2^-1 = 0.5
    0 × 2^-2 = 0.0
    1 × 2^-3 = 0.125
    小数部分总和: 0.625000
【十进制】3.625

【二进制】0.1011
  整数部分转换 (0 → 十进制):
    0 × 2^0 = 0
    整数部分总和: 0
  小数部分转换 (1011 → 十进制):
```

```
    1 × 2^-1 = 0.5
    0 × 2^-2 = 0.0
    1 × 2^-3 = 0.125
    1 × 2^-4 = 0.0625
    小数部分总和: 0.687500
【十进制】0.6875

【二进制】111.11
  整数部分转换 (111 → 十进制):
    1 × 2^2 = 4
    1 × 2^1 = 2
    1 × 2^0 = 1
    整数部分总和: 7
  小数部分转换 (11 → 十进制):
    1 × 2^-1 = 0.5
    1 × 2^-2 = 0.25
    小数部分总和: 0.750000
【十进制】7.75

【二进制】111111
  整数部分转换 (111111 → 十进制):
    1 × 2^5 = 32
    1 × 2^4 = 16
    1 × 2^3 = 8
    1 × 2^2 = 4
    1 × 2^1 = 2
    1 × 2^0 = 1
    整数部分总和: 63
【十进制】63
```

题目3：进制转换

题目描述

将下列二进制数转换为八进制和十六进制，八或十六进制转为二进制数： - 【二进制数】101110101，1101100.11 - 【八进制数】3756，415.213 - 【十六进制】C6F0，5AB.4D

代码实现

```python
def binary_to_octal(binary_str):
    if "." in binary_str:
        integer_part, fractional_part = binary_str.split(".", 1)
    else:
        integer_part = binary_str
        fractional_part = ""

    print(f"  二进制整数部分分组 ({integer_part}):")
    pad_length = (3 - len(integer_part) % 3) % 3
    padded_integer = "0" * pad_length + integer_part
    print(f"    补零后: {padded_integer}")

    octal_integer = []
    for i in range(0, len(padded_integer), 3):
        group = padded_integer[i:i+3]
        octal_digit = str(int(group, 2))
        octal_integer.append(octal_digit)
        print(f"    分组 {group} → 八进制 {octal_digit}")

    octal_fractional = []
    if fractional_part:
        print(f"  二进制小数部分分组 ({fractional_part}):")
        pad_length = (3 - len(fractional_part) % 3) % 3
        padded_fractional = fractional_part + "0" * pad_length
        print(f"    补零后: {padded_fractional}")

        for i in range(0, len(padded_fractional), 3):
            group = padded_fractional[i:i+3]
            octal_digit = str(int(group, 2))
            octal_fractional.append(octal_digit)
            print(f"    分组 {group} → 八进制 {octal_digit}")

    result = "".join(octal_integer)
    if octal_fractional:
        result += "." + "".join(octal_fractional)

    return result


def binary_to_hex(binary_str):
    hex_chars = "0123456789ABCDEF"
    if "." in binary_str:
        integer_part, fractional_part = binary_str.split(".", 1)
    else:
        integer_part = binary_str
        fractional_part = ""

    print(f"  二进制整数部分分组 ({integer_part}):")
    pad_length = (4 - len(integer_part) % 4) % 4
    padded_integer = "0" * pad_length + integer_part
    print(f"    补零后: {padded_integer}")

    hex_integer = []
    for i in range(0, len(padded_integer), 4):
        group = padded_integer[i:i+4]
        hex_digit = hex_chars[int(group, 2)]
        hex_integer.append(hex_digit)
        print(f"    分组 {group} → 十六进制 {hex_digit}")

    hex_fractional = []
```

```
    if fractional_part:
        print(f"  二进制小数部分分组 ({fractional_part}):")
        pad_length = (4 - len(fractional_part) % 4) % 4
        padded_fractional = fractional_part + "0" * pad_length
        print(f"    补零后: {padded_fractional}")

        for i in range(0, len(padded_fractional), 4):
            group = padded_fractional[i:i+4]
            hex_digit = hex_chars[int(group, 2)]
            hex_fractional.append(hex_digit)
            print(f"    分组 {group} → 十六进制 {hex_digit}")

    result = "".join(hex_integer)
    if hex_fractional:
        result += "." + "".join(hex_fractional)

    return result


def octal_to_binary(octal_str):
    octal_to_bin = {
        "0": "000", "1": "001", "2": "010", "3": "011",
        "4": "100", "5": "101", "6": "110", "7": "111"
    }

    if "." in octal_str:
        integer_part, fractional_part = octal_str.split(".", 1)
    else:
        integer_part = octal_str
        fractional_part = ""

    print(f"  八进制整数部分转换 ({integer_part}):")
    binary_integer = []
    for digit in integer_part:
        binary = octal_to_bin[digit]
        binary_integer.append(binary)
        print(f"    {digit} → {binary}")

    binary_fractional = []
    if fractional_part:
        print(f"  八进制小数部分转换 ({fractional_part}):")
        for digit in fractional_part:
            binary = octal_to_bin[digit]
            binary_fractional.append(binary)
            print(f"    {digit} → {binary}")

    result = "".join(binary_integer).lstrip("0") or "0"
    if binary_fractional:
        result += "." + "".join(binary_fractional).rstrip("0")

    return result


def hex_to_binary(hex_str):
    hex_to_bin = {
        "0": "0000", "1": "0001", "2": "0010", "3": "0011",
        "4": "0100", "5": "0101", "6": "0110", "7": "0111",
        "8": "1000", "9": "1001", "A": "1010", "B": "1011",
        "C": "1100", "D": "1101", "E": "1110", "F": "1111",
        "a": "1010", "b": "1011", "c": "1100", "d": "1101",
        "e": "1110", "f": "1111"
    }

    if "." in hex_str:
        integer_part, fractional_part = hex_str.split(".", 1)
    else:
        integer_part = hex_str
        fractional_part = ""

    print(f"  十六进制整数部分转换 ({integer_part}):")
    binary_integer = []
    for digit in integer_part:
        binary = hex_to_bin[digit]
        binary_integer.append(binary)
        print(f"    {digit} → {binary}")

    binary_fractional = []
    if fractional_part:
        print(f"  十六进制小数部分转换 ({fractional_part}):")
        for digit in fractional_part:
            binary = hex_to_bin[digit]
            binary_fractional.append(binary)
            print(f"    {digit} → {binary}")

    result = "".join(binary_integer).lstrip("0") or "0"
    if binary_fractional:
        result += "." + "".join(binary_fractional).rstrip("0")

    return result
```

运行结果

```
【二进制】101110101
  二进制整数部分分组 (101110101):
    补零后: 101110101
    分组 101 → 八进制 5
    分组 110 → 八进制 6
    分组 101 → 八进制 5
【八进制】565
  二进制整数部分分组 (101110101):
    补零后: 000101110101
    分组 0001 → 十六进制 1
    分组 0111 → 十六进制 7
    分组 0101 → 十六进制 5
【十六进制】175


【二进制】1101100.11
  二进制整数部分分组 (1101100):
```

```
    补零后: 001101100
    分组 001 → 八进制 1
    分组 101 → 八进制 5
    分组 100 → 八进制 4
  二进制小数部分分组 (11):
    补零后: 110
    分组 110 → 八进制 6
【八进制】154.6
  二进制整数部分分组 (1101100):
    补零后: 01101100
    分组 0110 → 十六进制 6
    分组 1100 → 十六进制 C
  二进制小数部分分组 (11):
    补零后: 1100
    分组 1100 → 十六进制 C
【十六进制】6C.C

【八进制】3756
  八进制整数部分转换 (3756):
    3 → 011
    7 → 111
    5 → 101
    6 → 110
【二进制】11111101110

【八进制】415.213
  八进制整数部分转换 (415):
    4 → 100
    1 → 001
    5 → 101
  八进制小数部分转换 (213):
    2 → 010
    1 → 001
    3 → 011
【二进制】100001101.010001011

【十六进制】C6F0
  十六进制整数部分转换 (C6F0):
    C → 1100
    6 → 0110
    F → 1111
    0 → 0000
【二进制】1100011011110000

【十六进制】5AB.4D
  十六进制整数部分转换 (5AB):
    5 → 0101
    A → 1010
    B → 1011
  十六进制小数部分转换 (4D):
    4 → 0100
    D → 1101
【二进制】10110101011.01001101
```

题目4：Unicode编号和UTF-8编码

题目描述

查找下列字符的Unicode编号和UTF-8编码： - Python语言基础与人工智能应用

代码实现

```python
def unicode_utf8_analysis(text):
    print(f"  字符 | Unicode编号 | UTF-8编码")
    print(f"  {'-'*40}")
    for char in text:
        unicode_code = ord(char)
        utf8_bytes = char.encode('utf-8')
        utf8_hex = " ".join(f"{b:02X}" for b in utf8_bytes)
        print(f"  '{char}' | U+{unicode_code:04X} | {utf8_hex}")
```

运行结果

```
【字符串】Python语言基础与人工智能应用
  字符 | Unicode编号 | UTF-8编码
  ----------------------------------------
  'P' | U+0050 | 50
  'y' | U+0079 | 79
  't' | U+0074 | 74
  'h' | U+0068 | 68
  'o' | U+006F | 6F
  'n' | U+006E | 6E
```

```
'语' | U+8BED | E8 AF AD
'言' | U+8A00 | E8 A8 80
'基' | U+57FA | E5 9F BA
'础' | U+7840 | E7 A1 80
'与' | U+4E0E | E4 B8 8E
'人' | U+4EBA | E4 BA BA
'工' | U+5DE5 | E5 B7 A5
'智' | U+667A | E6 99 BA
'能' | U+80FD | E8 83 BD
'应' | U+5E94 | E5 BA 94
'用' | U+7528 | E7 94 A8
```